

Serverless Mass Emailing System using AWS Lambda & SES

Sem End Project Report

Cloud and Serverless Computing

Department of Computer Science and Engineering

By

2200031356

Section – 33

Advanced

Under the supervision of

Mr. Rajesh Bingu



Koneru Lakshmaiah Education

Foundation

(Deemed to be University estd., u/s 3 of UGC Act 1956)

Green Fields, Vaddeswaram, Guntur (Dist.), Andhra Pradesh – 522302

CONTENTS

I.	Abstract.....	04
II.	Introduction.....	05
III.	Literature Review.....	06
IV.	Project Objectives and Scope	
	4.1 Problem Statement.....	07
V.	Technical Implementation.....	08
	5.1 Data Gathering.....	08
	5.2 Creativity & Originality.....	09
VI.	Analysis & Problem-Solving.....	10
VII.	Discussions & Results.....	42
VIII.	Conclusion.....	43
IX.	Future Work.....	43
X.	References.....	44

LIST OF FIGURES

Figure No	Title	Page Number
1	Creating an S3 Bucket and Uploading Email Template Objects	11-12
2	Configuring Amazon SES (Sending Email Verification)	12-14
3	Creating and Verifying Identities in Amazon SES	14-17
4	Creating an AWS Lambda Function & Testing the Event	18-21
5	Creating an IAM Role and IAM Policy for Lambda Execution and SES Access	21-24
6	Creating EventBridge Rule to Trigger Lambda Based on a Schedule	24-28
7	Viewing Logs in Amazon CloudWatch for Lambda Execution Results	28-29
8	Monitoring SES Sending Metrics and CloudWatch Dashboards	29-31

Abstract

This project presents a Serverless Mass Emailing System built on Amazon Web Services (AWS), designed to automate and scale bulk email communication with zero infrastructure management. By adopting a serverless approach, the system enables fast, reliable, and cost-effective email delivery, making it ideal for marketing campaigns, customer notifications, and automated outreach.

The process begins with users uploading a CSV file containing email addresses and metadata to an Amazon S3 bucket. This triggers an event in Amazon EventBridge, which automatically invokes an AWS Lambda function. The Lambda function parses the file and sends customized emails to each recipient using Amazon Simple Email Service (SES)—a robust and scalable email platform.

Security and access control are handled through AWS IAM roles and policies, ensuring that each component has only the permissions it needs. This principle of least privilege enhances the system's security posture. Amazon CloudWatch is used to monitor Lambda executions, track email delivery outcomes, and diagnose issues with logs and metrics.

The solution is fully event-driven and eliminates operational overhead, offering a highly scalable and maintainable architecture. Overall, this solution demonstrates a complete event-driven, serverless architecture that is both scalable and operationally efficient—suitable for email marketing, transactional notifications, and customer engagement automation. This project exemplifies the power of cloud-native design, offering a template for scalable, resilient, and fully automated communication systems in the modern digital landscape.

Introduction

This project focuses on building a serverless, scalable, and automated mass emailing system using Amazon Web Services (AWS). It is designed to efficiently manage and send bulk emails without the need for backend server provisioning or maintenance. Leveraging AWS's serverless capabilities, the system delivers high availability, low operational overhead, and cost-effective performance—ideal for use cases such as marketing campaigns, transactional notifications, and customer updates.

The workflow begins when users upload a CSV file containing recipient information to an Amazon S3 bucket. This upload triggers an event captured by Amazon EventBridge, which then invokes an AWS Lambda function. The Lambda function processes the file, parses email addresses, and sends personalized messages using Amazon Simple Email Service (SES), enabling fast and reliable email delivery at scale.

To maintain security and controlled access, AWS Identity and Access Management (IAM) roles and policies are configured to follow the principle of least privilege. Each service involved is granted only the permissions necessary to perform its specific task, reducing risk and ensuring compliance with security best practices.

Amazon CloudWatch is integrated into the architecture to provide comprehensive logging, real-time metrics, and alerting capabilities. It helps monitor Lambda executions, track email delivery outcomes, and quickly identify any system issues. Overall, this event-driven serverless solution showcases the power of AWS in building robust, secure, and scalable email automation systems.

Literature Review

Overview of Mass Emailing System

Mass emailing systems are essential tools for organizations to communicate with large audiences, used for promotional campaigns, newsletters, customer notifications, and transactional updates. Traditional systems required dedicated servers, complex configurations, and ongoing maintenance, making scalability and reliability challenging, especially for growing businesses.

Cloud computing has revolutionized mass emailing by offering scalable, automated solutions that reduce infrastructure management. Cloud-based platforms like SendGrid, Mailchimp, and Amazon SES simplify email processes with APIs, built-in tracking, and analytics, allowing for seamless scaling and high availability.

Serverless architecture has further enhanced mass emailing systems by using event-driven models. Services like AWS Lambda enable backend logic execution only when triggered, eliminating the need for continuous server management and reducing operational costs. This model supports dynamic workloads and integrates with services like Amazon S3 and EventBridge for real-time, automated email handling.

Security and compliance are crucial in modern mass emailing systems. To meet privacy and anti-spam regulations (e.g., GDPR, CAN-SPAM), cloud providers offer solutions like IAM roles, data encryption, and policy management. These features ensure secure, compliant, and efficient email communication workflows in today's digital environment.

Project Objectives and Scope

Problem Statement:

In today's digital communication landscape, organizations need a reliable and efficient way to send mass emails to large audiences. Traditional emailing systems often require significant infrastructure management, server maintenance, and complex configurations, which can lead to scalability issues, high operational costs, and delays in email delivery. Additionally, ensuring email deliverability, tracking engagement, and maintaining compliance with data privacy regulations adds further complexity.

This project aims to address these challenges by developing a serverless mass emailing system using AWS Lambda and Amazon Simple Email Service (SES). The objective is to automate the entire email sending process, from capturing email requests and storing data to sending emails in bulk and tracking delivery statuses—while ensuring high availability, scalability, and cost-efficiency. By leveraging AWS services like Lambda, SES, API Gateway, DynamoDB, and SNS, the system will dynamically scale to handle high-volume email campaigns without the need for continuous server management.

Furthermore, the architecture will enable future enhancements, such as personalized email content, automated A/B testing, and advanced analytics. With tools like Amazon Pinpoint or CloudWatch, the system can track user engagement and optimize email campaigns. The goal is to deliver a seamless, automated, and secure mass emailing experience that is scalable for organizations of all sizes—from startups to large enterprises—while maintaining compliance with privacy regulations like GDPR and CAN-SPAM.

Technical Implementation

5.1 Data Gathering

Data gathering is critical for building an efficient and personalized mass emailing system. The project involves collecting structured data related to recipients, email content, delivery preferences, and engagement metrics to ensure smooth execution and future personalization.

- **Data Sources:** Data is sourced primarily from CSV files uploaded by users to Amazon S3. These files include:
 - **Recipient Data:** Names, email addresses, segments/tags (e.g., “premium”, “trail user”), and preferences.
 - **Campaign Data:** Subject lines, body content templates, sending time, and target groups.
 - **Interaction Data:** (Optional for analytics) Open rates, click-through rates, and bounce reports from Amazon SES or CloudWatch logs.
- **Data Preparation and Storage:** Once the CSV is uploaded, an AWS Lambda function parses and validates the data. Cleaned and structured data is optionally stored in Amazon DynamoDB for tracking past campaigns or managing recurring recipients. Amazon S3 holds the raw uploaded files, and API Gateway serves as a secure entry point for authenticated users initiating email campaigns.
- **Data Privacy and Ethics:** The system respects privacy standards by encrypting sensitive information, securing access via IAM roles, and ensuring compliance with GDPR and CAN-SPAM. Transmission of data between services is protected via HTTPS and IAM policies ensure least-privilege access control.

5.2 Creativity & Originality

While the system uses AWS-managed services, its uniqueness lies in its event-driven design and flexibility to handle various email campaign needs dynamically and securely.

- **Serverless Architecture:** The architecture uses Amazon S3 for file uploads, EventBridge for event triggering, AWS Lambda for data processing and business logic, and Amazon SES for email delivery—all without maintaining any traditional servers.
- **Dynamic Personalization:** Personalized emails can be generated using placeholders (e.g., {{Name}}, {{PromoCode}}) within the email templates that get replaced dynamically by Lambda, allowing hyper-personalized content per recipient.
- **Smart Email Routing:** Campaigns can be segmented by user attributes (like region, plan type), and separate Lambda functions can route content accordingly. This modular approach enhances performance and delivery relevance.
- **Enhanced Features:**
 - **Real-time Monitoring:** Integration with Amazon CloudWatch for logging and alerting when email sending fails or metrics drop.
 - **Throttling & Retry Logic:** Built-in logic to respect SES sending quotas and automatically retry failed sends.
 - **A/B Testing Ready:** Email variations can be processed using separate CSV tags, allowing future implementation of A/B test campaigns.
- **Scalability & Cost-Efficiency:** Serverless services allow the system to handle thousands of emails with minimal cost during idle periods, making it suitable for startups and enterprises alike.

Analysis & Problem-Solving

The aim is to develop an automated, secure, and real-time mass emailing solution that can process large email lists and send personalized messages quickly and reliably.

Key Components Involved:

1. User Data:

- Understanding the target audience through names, email addresses, location, and custom tags (e.g., “newsletter_subscriber”, “event_attendee”).

2. Menu & Item Data:

- Designing email bodies that include placeholders for personalization, template variations, and localized content support.

3. User-Item Interactions:

- Optional future integration with Amazon Pinpoint or SES event tracking to monitor open rates, bounces, and click-through metrics.

4. Real-Time Personalization:

- Automated execution using an event-driven workflow:
- CSV upload → EventBridge → Lambda → SES
- CloudWatch tracks every step and sends alerts for anomalies.

5. Security & Compliance:

- IAM for strict access control, SES domain verification for email authenticity, and optional anonymization of logs for GDPR compliance.

Problem-Solving Process

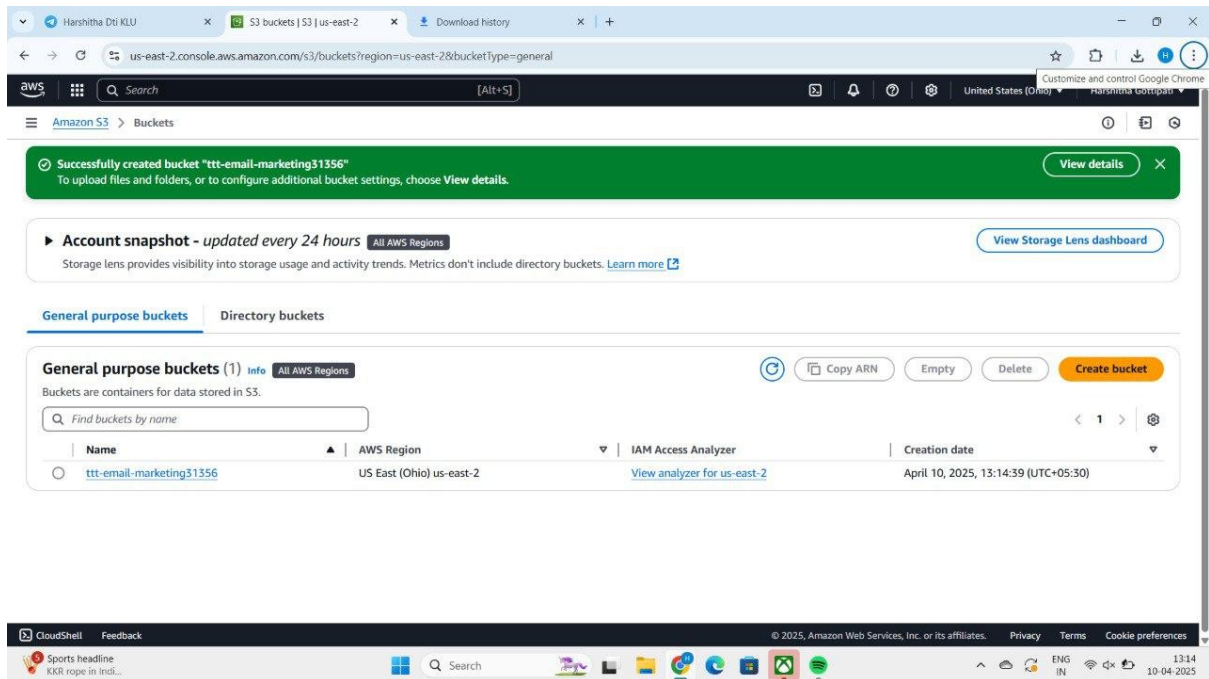


Fig1: Navigate to Amazon S3, click on “Create bucket”, and give your bucket a name (e.g., ttt-email-marketing31356). Choose the region (e.g., US East - Ohio), and click Create bucket. This bucket will be used to upload your email contact list in CSV format.

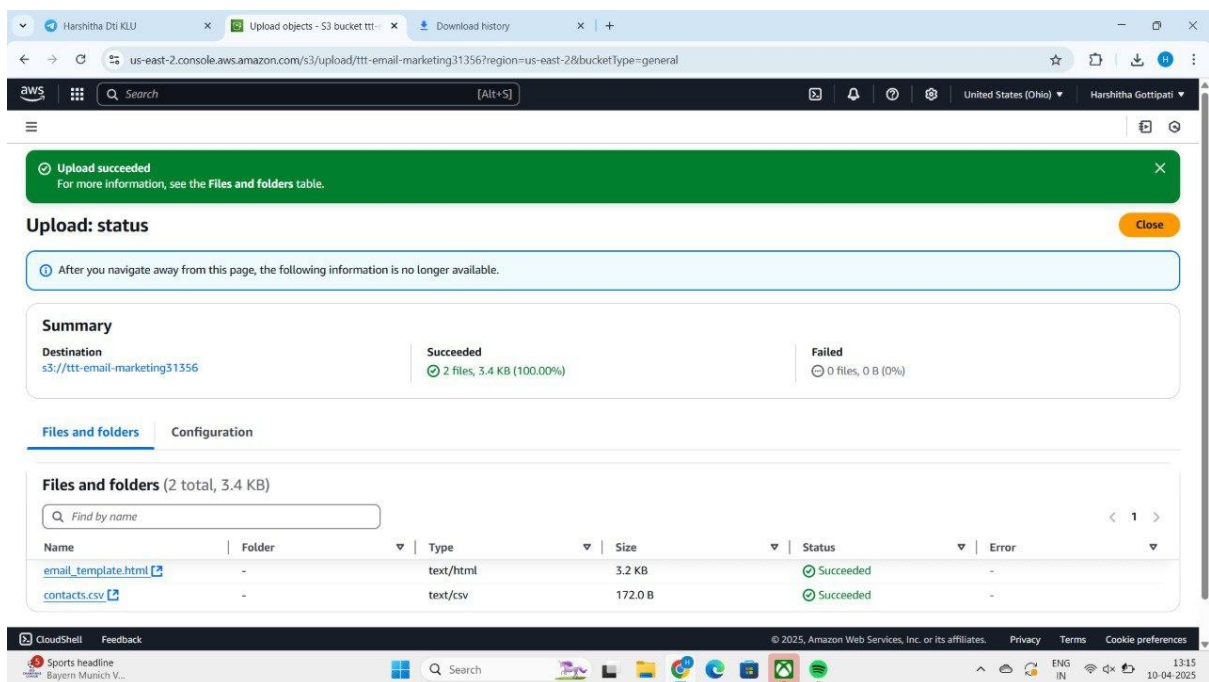


Fig2: Navigate to your created S3 bucket (e.g., ttt-email-marketing31356). Click “Upload”, add your email template (email_template.html) and contact list (contacts.csv), then click “Upload”. These files will be used by the Lambda function to send emails via SES.

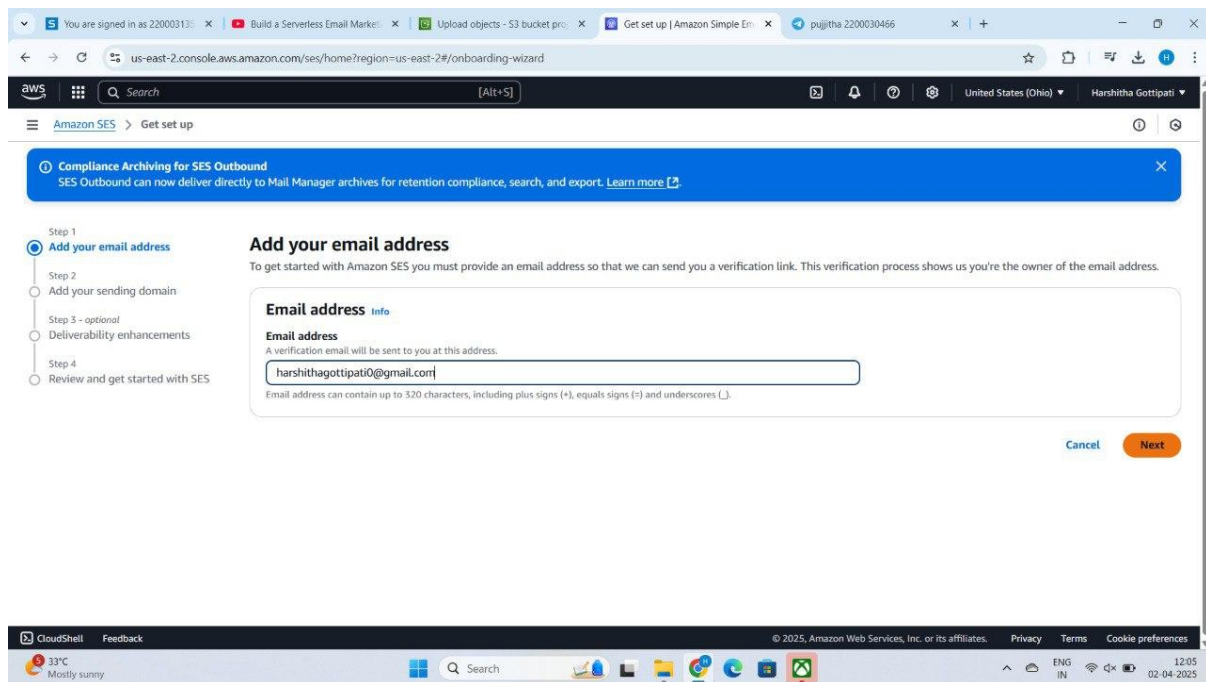


Fig3: Navigate to the [Amazon SES console](#). Under “Get set up”, enter your email address (e.g., [johnsmith@gmail.com](#)) and click Next. A verification email will be sent to this address. Open the email and click the verification link to confirm ownership. This verified address will be used to send test emails.

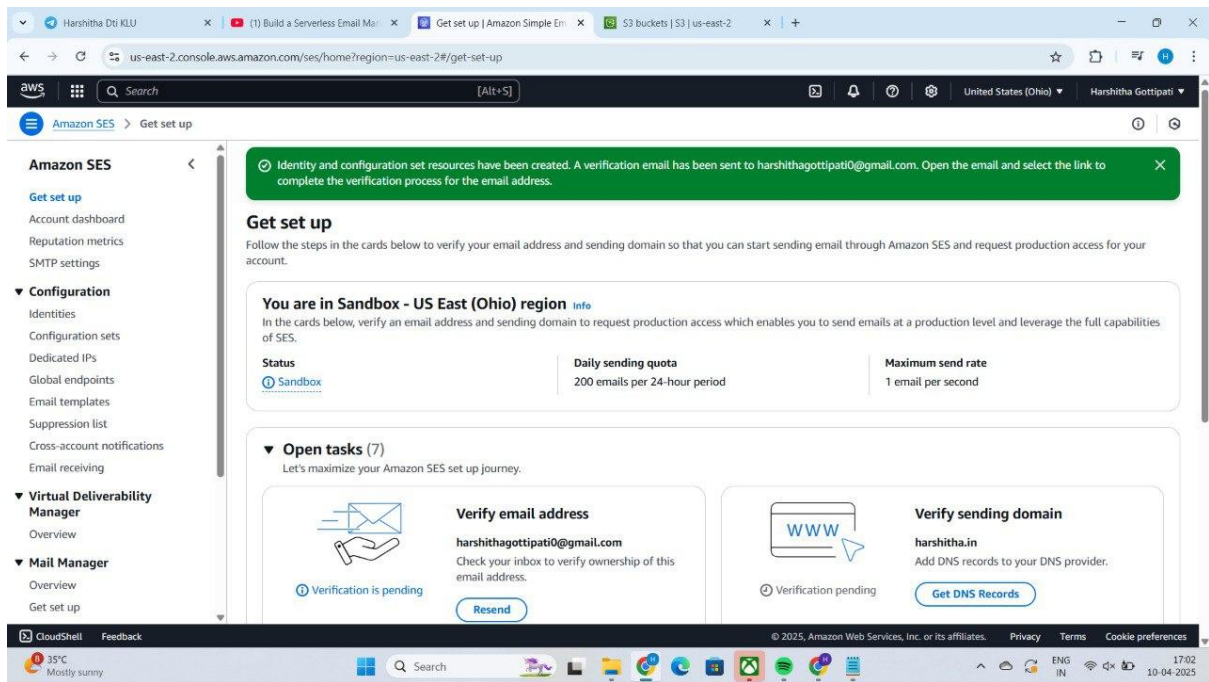


Fig4: After submitting your email for verification, Amazon SES will show the Sandbox environment with a daily sending quota (e.g., 200 emails/day). You must verify your email (check your inbox and click the verification link). Until verification is complete, email sending is restricted. You can also verify a sending domain if needed.

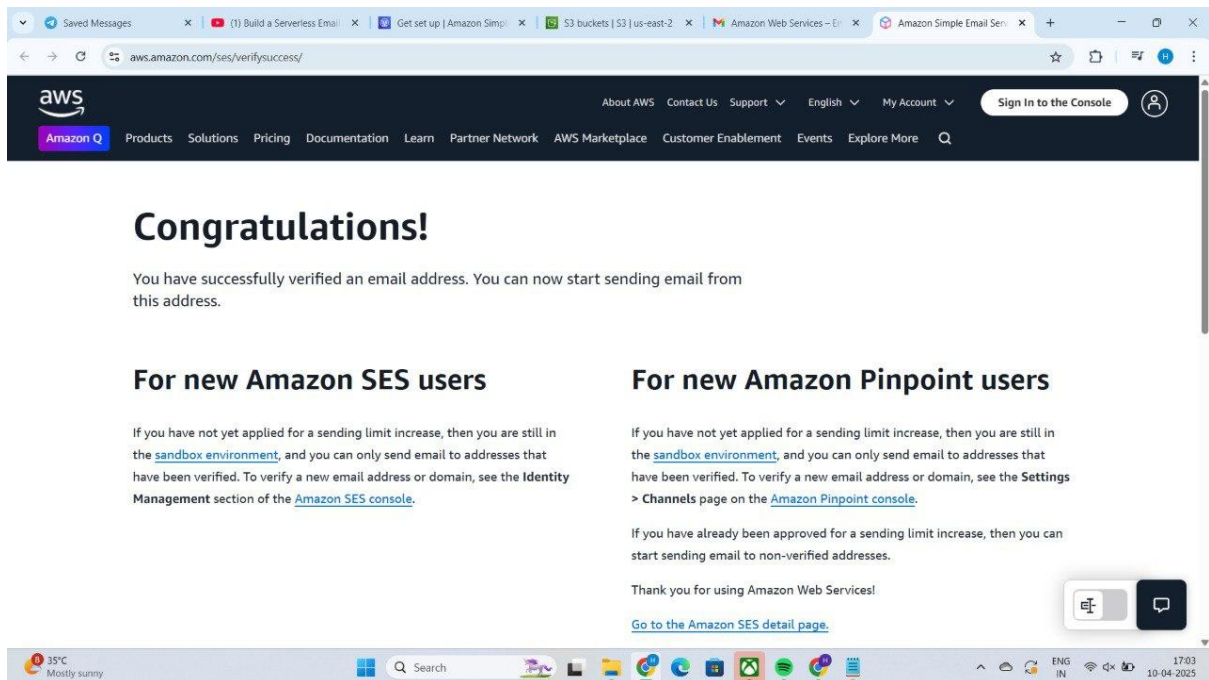


Fig5: Once you click the verification link in your inbox, you'll see a success message confirming your email is verified. You can now send emails from this address using Amazon SES (within the sandbox environment for now).

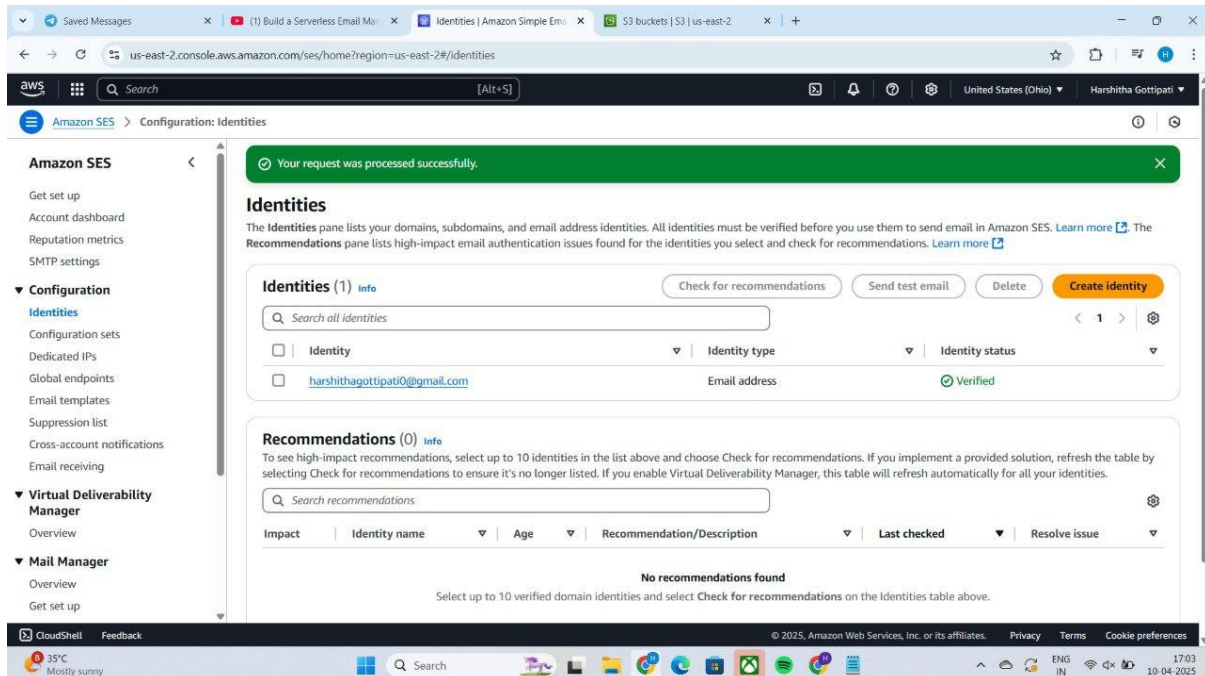


Fig6: Go to Amazon SES > Configuration > Identities. You'll now see your email (johnsmith@gmail.com) marked as Verified. This confirms it's ready for sending emails via SES.

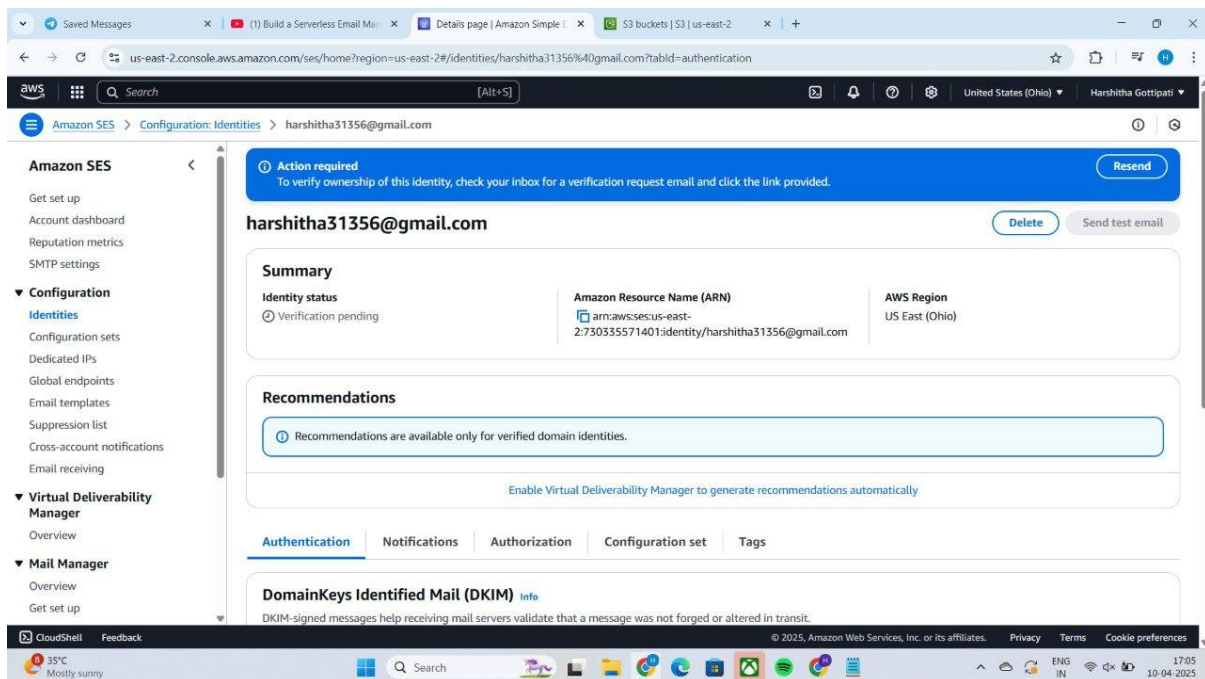


Fig7: Go to Amazon SES > Configuration > Identities. Your email (e.g., alice123@gmail.com) shows "Verification pending". Check your email inbox for a verification message from AWS, and click the link inside to complete the verification. You can click "Resend" if you didn't receive it.

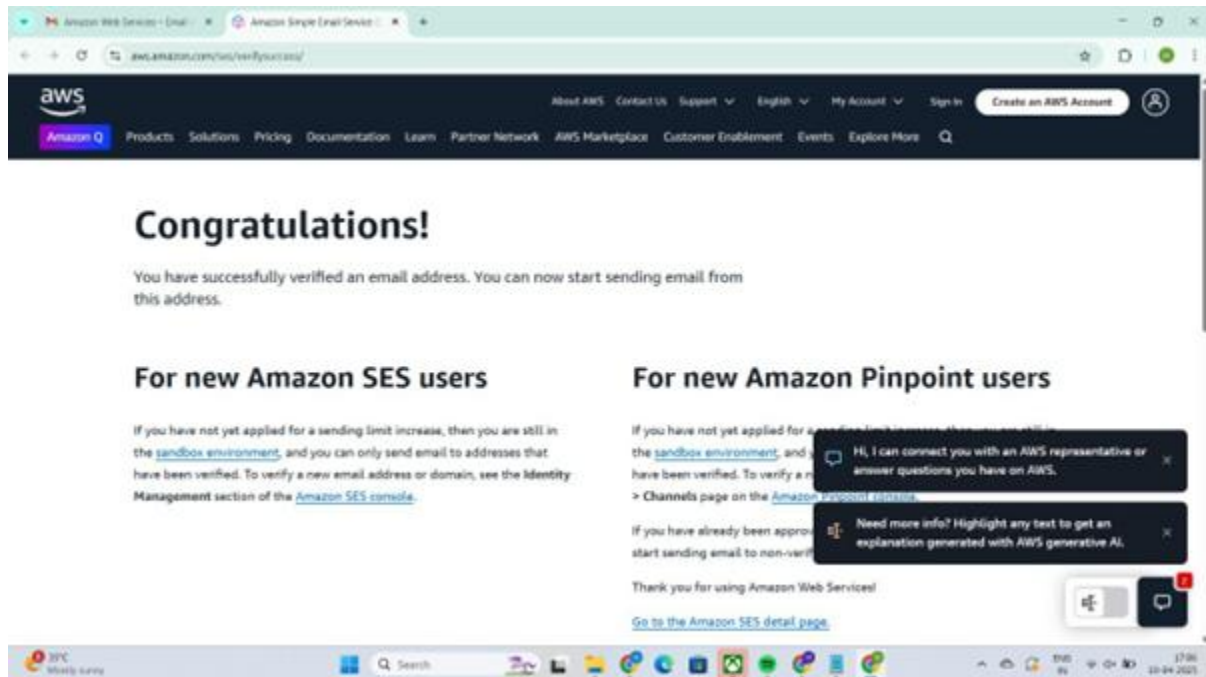


Fig 8: After clicking the verification link, you'll be redirected to a confirmation page saying "Congratulations! You have successfully verified an email address." This means your email is now approved to send messages via Amazon SES. You're still in the sandbox environment, so you can only email verified addresses unless you request a production access.

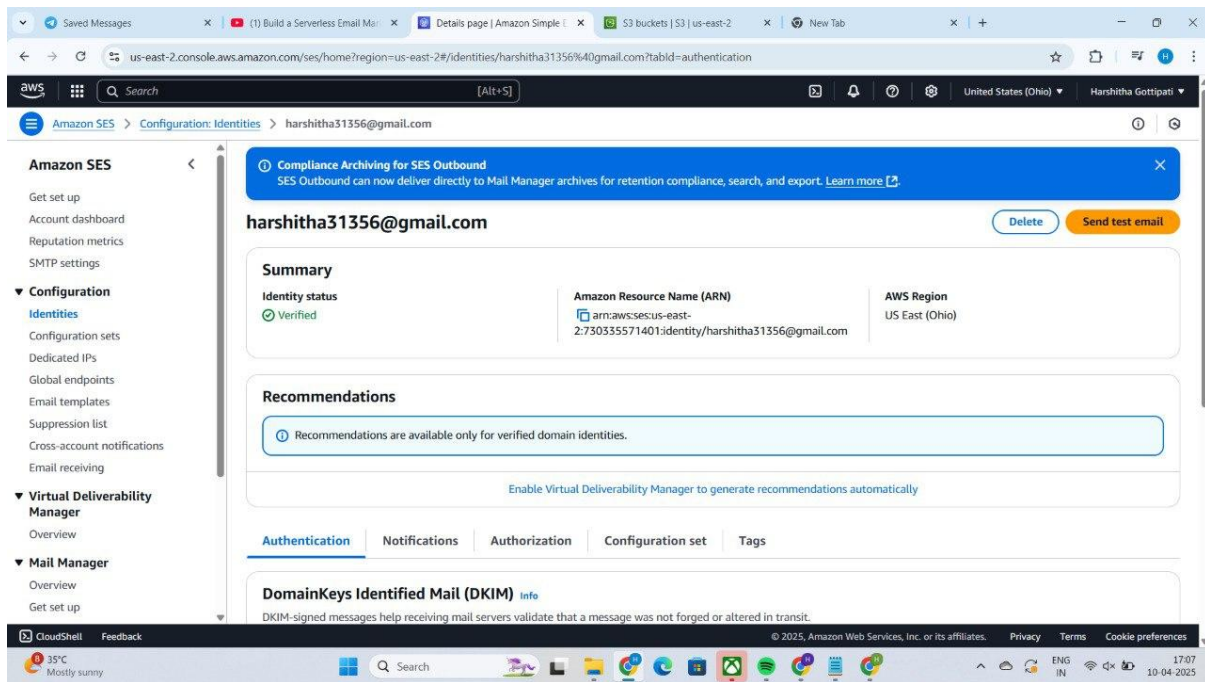


Fig 9: Return to Amazon SES > Configuration > Identities. Your email (e.g., alice123@gmail.com) now shows “Verified” as the identity status. This means you can start sending test emails using this verified email address. Click “Send test email” (top right) to try sending one via the SES console.

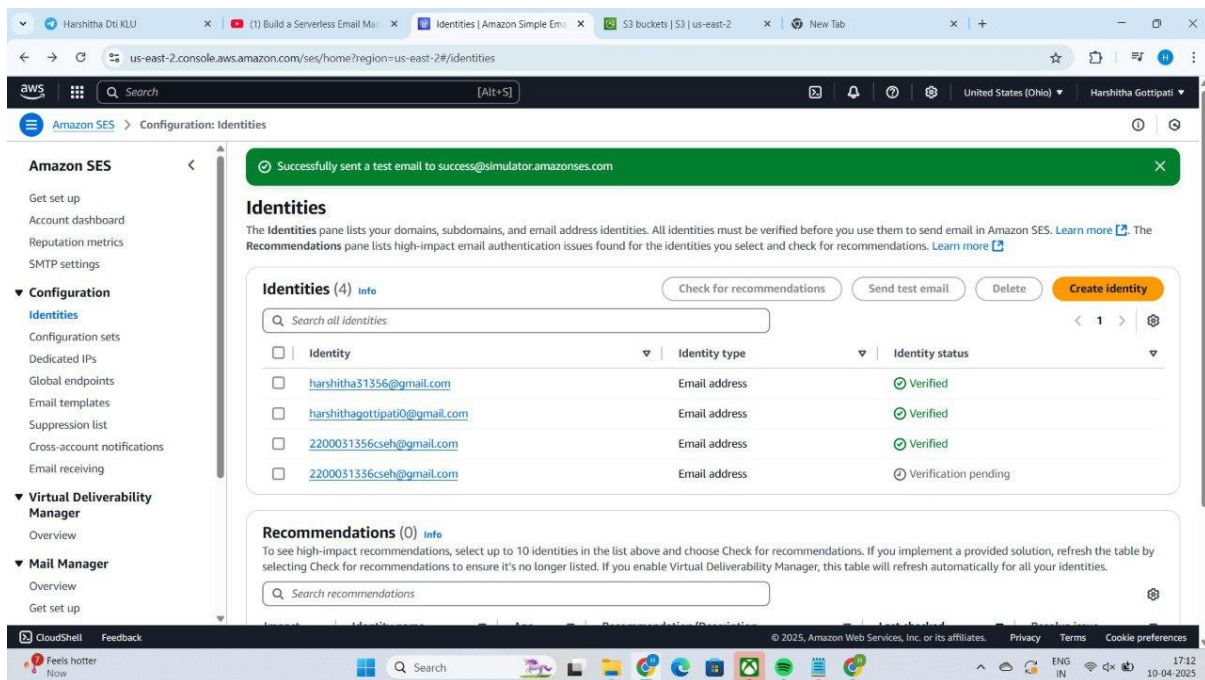


Fig 10: This image confirms that multiple email addresses have been added under Amazon SES > Configuration > Identities:

- **Verified:**

johnsmith123@gmail.com

alice789@gmail.com

chef456@gmail.com

david678@gmail.com

- **Verification Pending:**

elisa912@gmail.com

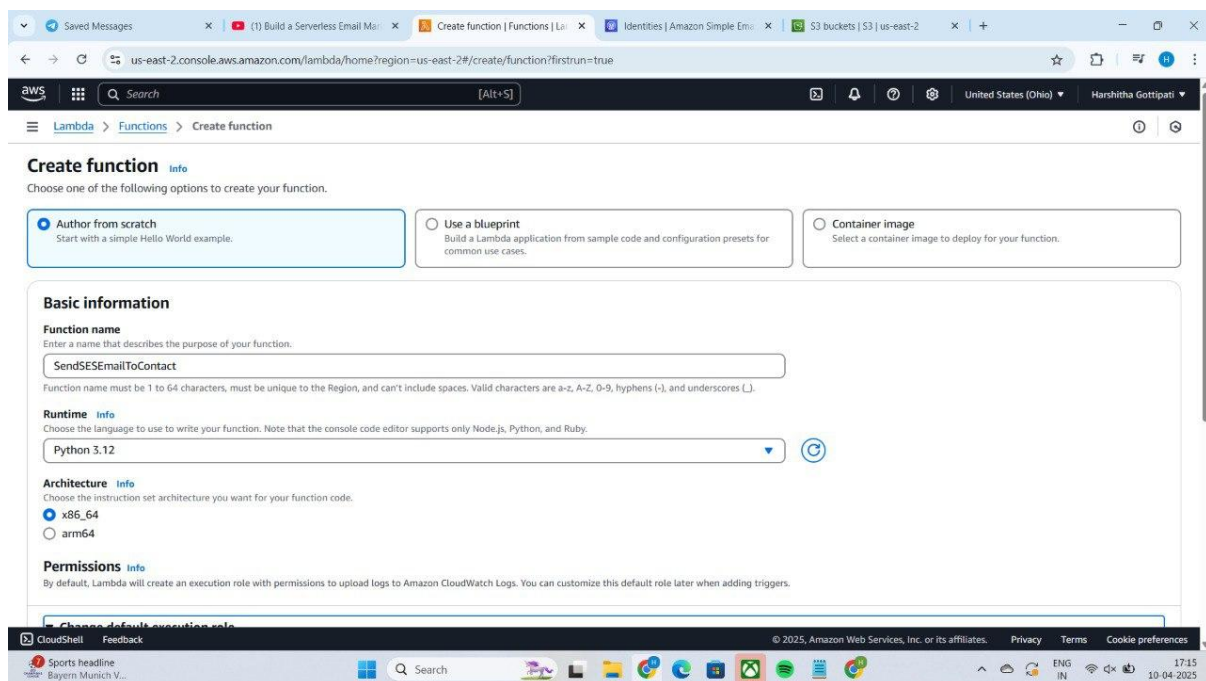


Fig 11: Navigate to the AWS Lambda console. Click “Create function”, choose “Author from scratch”, and enter a function name (e.g., SendSESEmailToContact). Select Python 3.12 as the runtime and architecture as x86_64. Leave permissions as default. Click Create function to proceed.

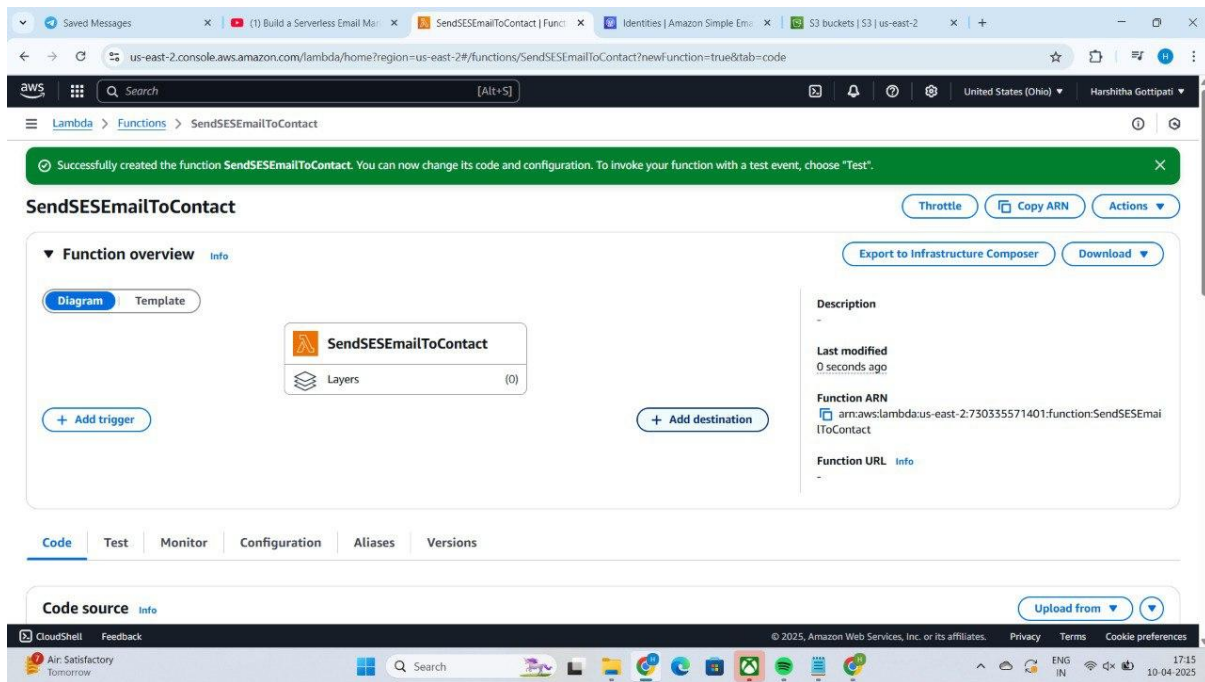


Fig 12: After creating the function SendSESEmailToContact, you'll see a confirmation banner and the function dashboard. From here, you can add triggers, upload code, and configure settings. Use the Code tab below to write or upload your email-sending logic using Python and the Boto3 library.

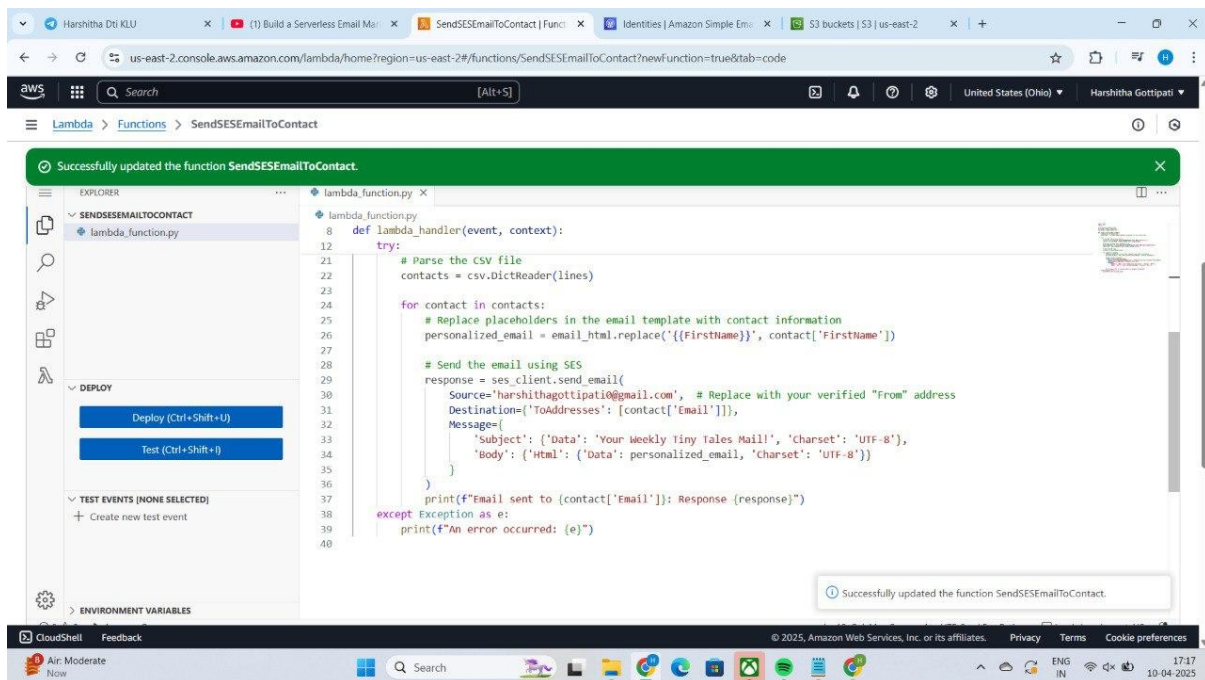


Fig 13: Manually wrote the Python code into the Code tab of your Lambda function. This script fetches the contacts.csv and email_template.html from your S3 bucket (e.g., ttt-email-marketing), personalizes the email for each contact, and sends it using Amazon SES. After pasting, click Deploy to make the function ready for execution.

Lambda Function Python Code

```
import boto3
import csv
s3_client = boto3.client('s3')
ses_client = boto3.client('ses')
def lambda_handler(event, context):
    bucket_name = 'ttt-email-marketing'
    try:
        csv_file = s3_client.get_object(Bucket=bucket_name, Key='contacts.csv')
        lines = csv_file['Body'].read().decode('utf-8').splitlines()
        email_template = s3_client.get_object(Bucket=bucket_name,
        Key='email_template.html')
        email_html = email_template['Body'].read().decode('utf-8')
        contacts = csv.DictReader(lines)
        for contact in contacts:
            personalized_email = email_html.replace('{{FirstName}}', contact['FirstName'])
            response = ses_client.send_email(
                Source='you@yourdomainname.com', # Replace with your verified "From"
                address
                Destination={'ToAddresses': [contact['Email']]},
                Message={
                    'Subject': {'Data': 'Your Weekly Tiny Tales Mail!', 'Charset': 'UTF-8'},
                    'Body': {'Html': {'Data': personalized_email, 'Charset': 'UTF-8'}}
                }
            )
```

)

```
print(f"Email sent to {contact['Email']}: Response {response}")
```

except Exception as e:

```
print(f"An error occurred: {e}")
```

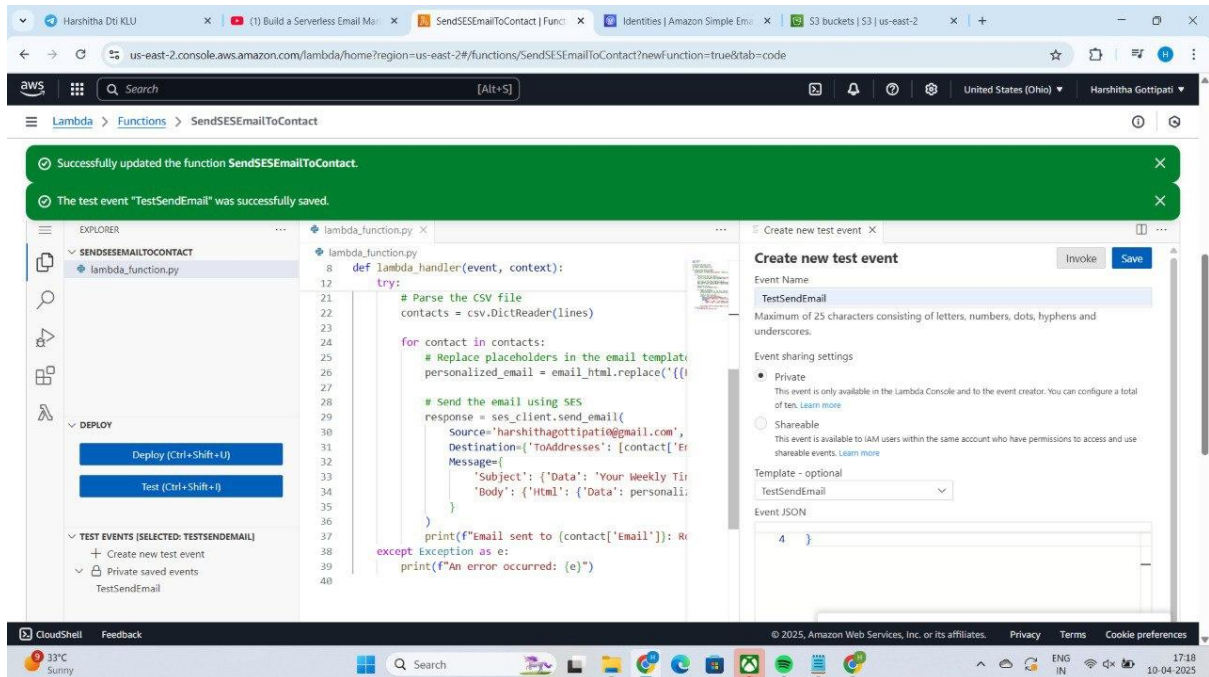


Fig 14: Name the event (e.g., TestSendEmails) and click Test. This triggers the Lambda function to send emails using the contacts and template from your S3 bucket.

Lambda Function Test Event Code

```
{
  "comment": "Generic test event for scheduled Lambda execution.
The function does not use this event data.",
  "test": true
}
```

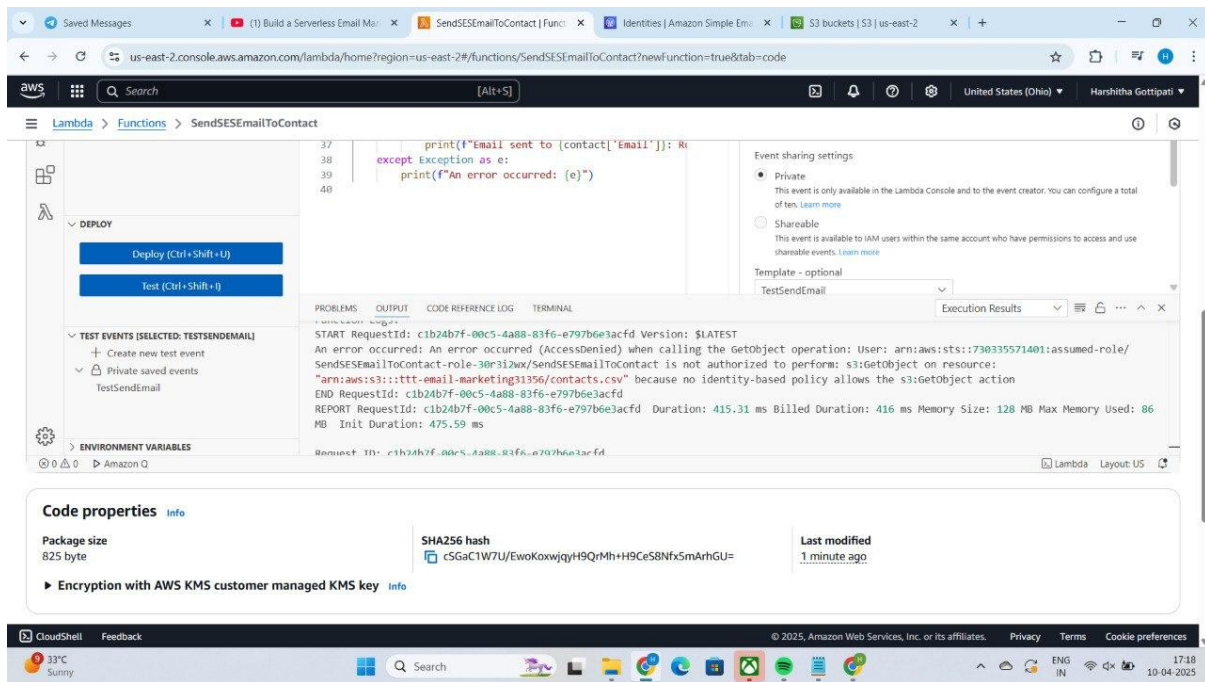


Fig 15: After testing the Lambda function, an AccessDenied error occurred. This means the Lambda function's IAM role does not have permission to perform s3:GetObject on the S3 bucket (ttt-email-marketing31356).

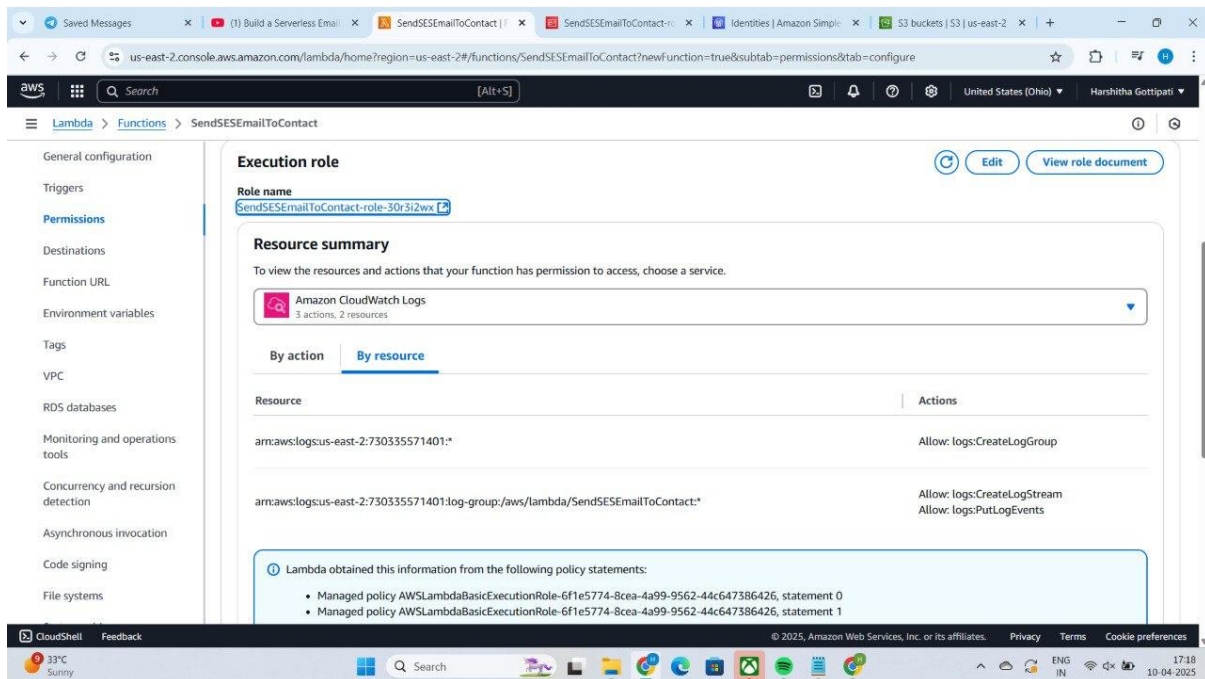


Fig 16: This screen shows the Lambda function's execution role `SendSESEmailToContact-role-30r3i2wx`, which currently has permissions only for Amazon CloudWatch Logs. It does not have access to Amazon S3, which caused the `AccessDenied` error.

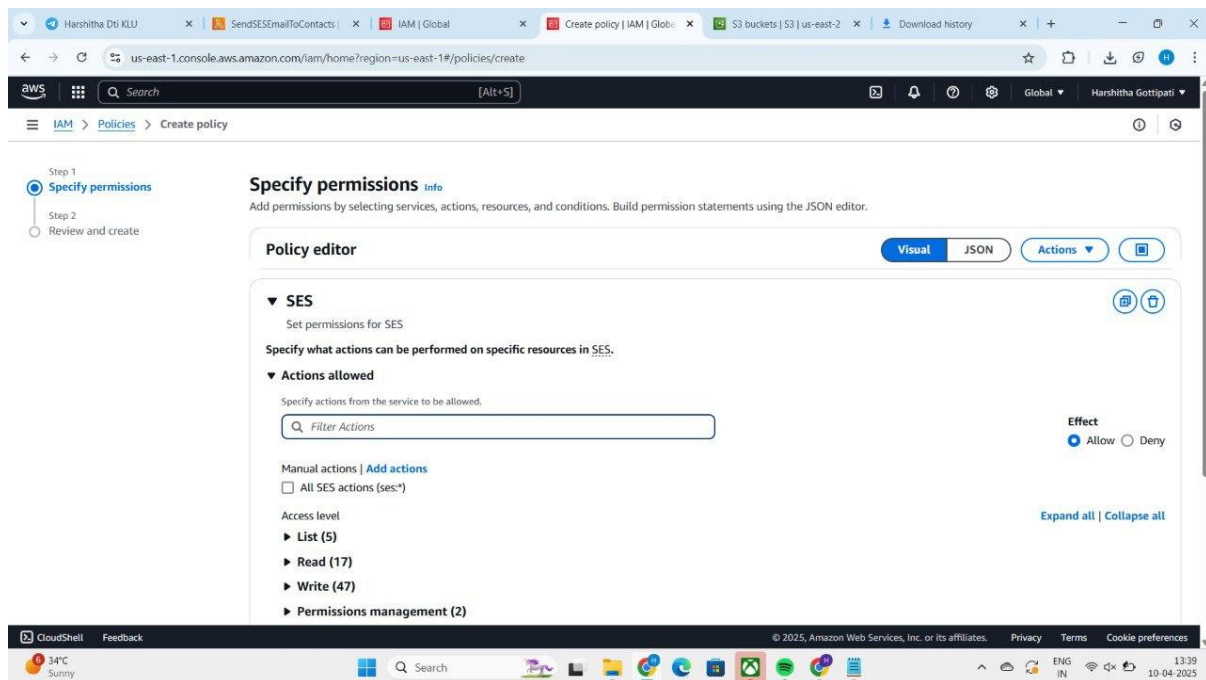


Fig 17: Creating the IAM Policy creation screen, where permissions are being defined for Amazon SES. You are selecting allowed actions (like `SendEmail`) to grant your Lambda function access to use SES.

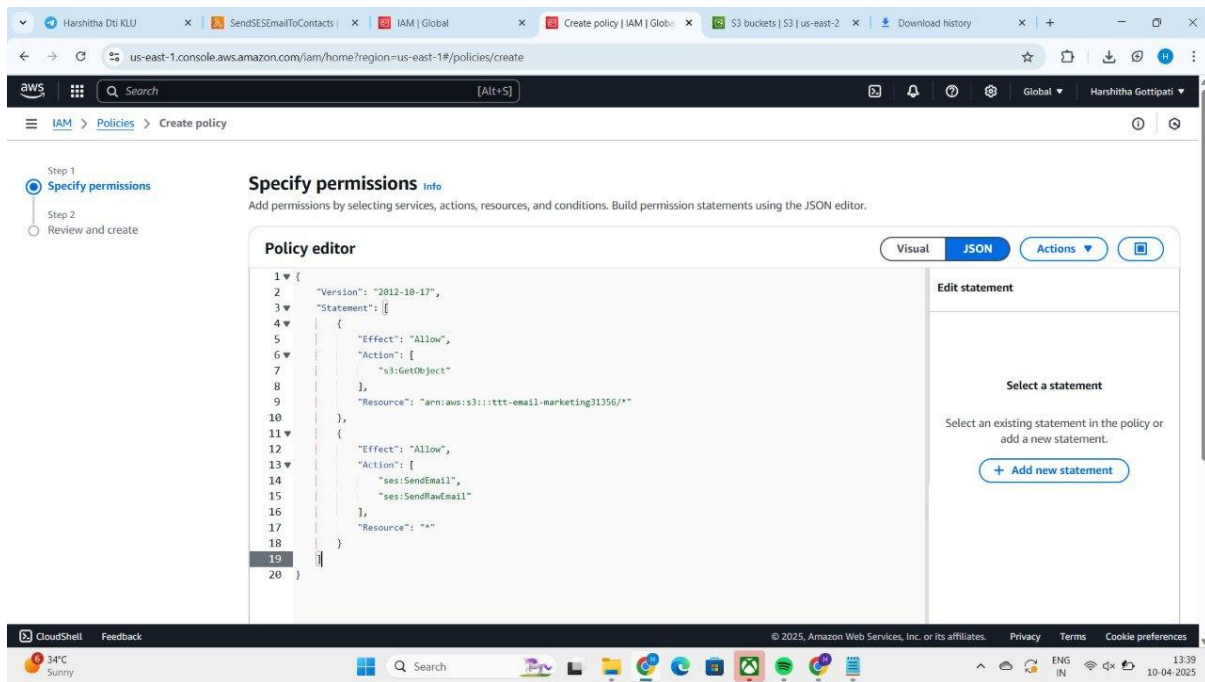


Fig 18: Navigate to the IAM console > Policies > Create policy. Switch to the JSON tab and paste the provided policy code. This allows the Lambda function to:

- **Read email templates and contacts from your S3 bucket (ttt-email-marketing)**
- **Send emails using Amazon SES (SendEmail and SendRawEmail actions) Click Next, review, and give the policy a name like SESS3AccessPolicy, then click Create policy.**

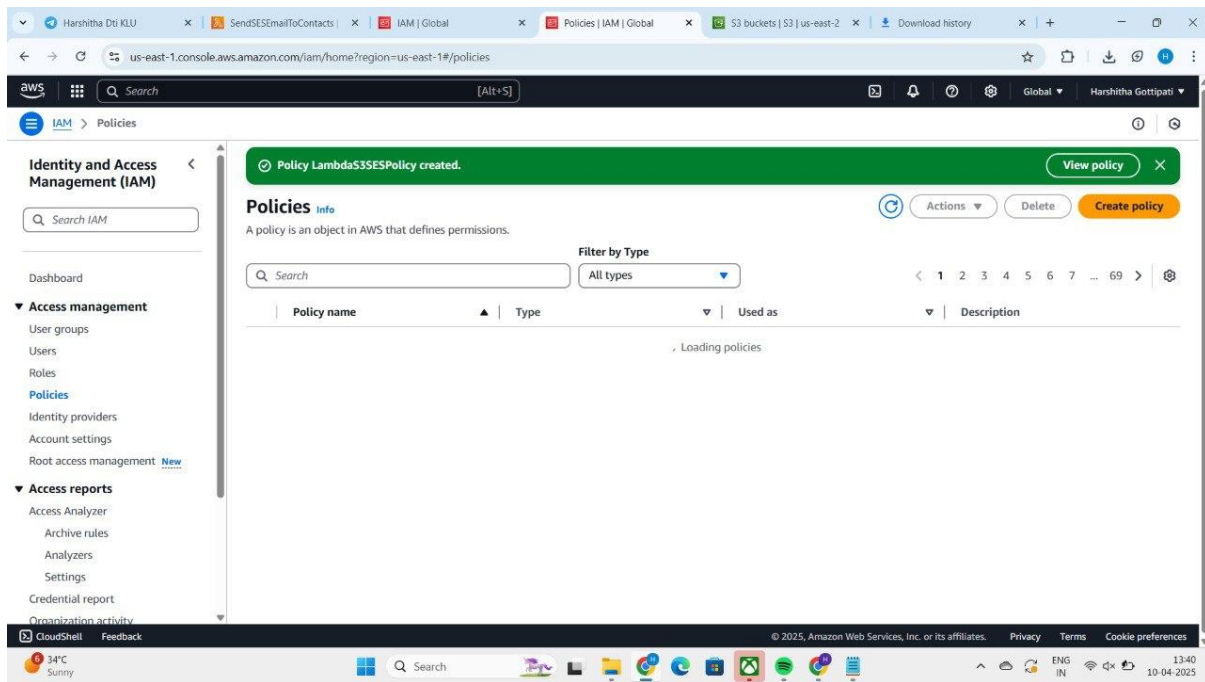


Fig 19: After pasting the policy JSON and reviewing it, you'll see a green banner confirming the policy (LambdaS3SESPolicy) was created successfully. You can now view this policy under IAM > Policies and attach it to your Lambda function's execution role to grant S3 read and SES send permissions.

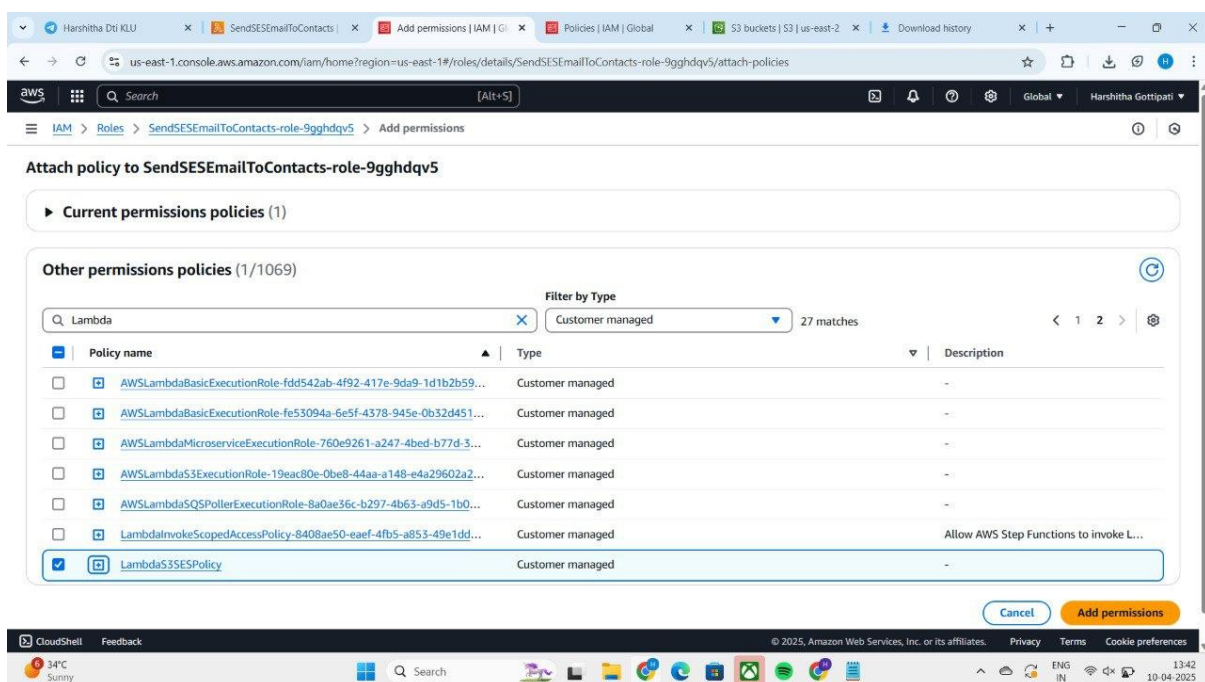


Fig 20: Navigate to IAM > Roles and select your Lambda role (SendSESEmailToContacts-role-9gghdqv5). Click “Add permissions”, search for your custom policy (LambdaS3SESPolicy), check the box, and click “Add permissions”. This grants the Lambda function access to read from S3 and send emails via SES.

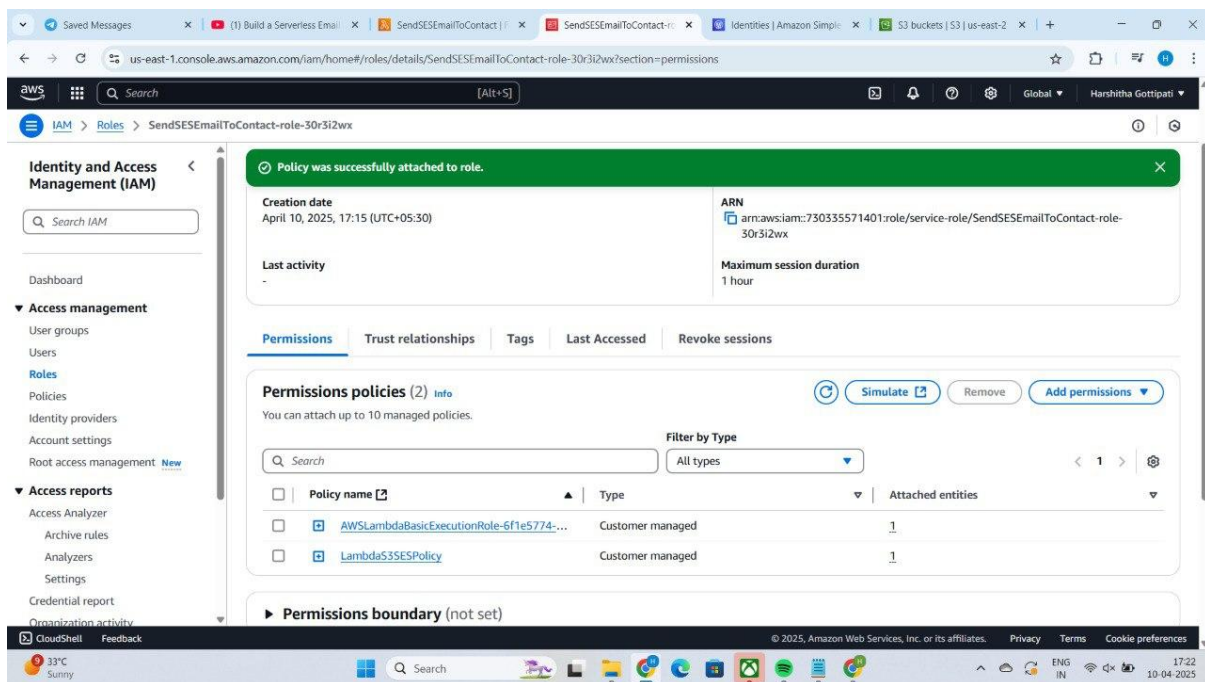


Fig 21: The custom policy LambdaS3SESPolicy is now attached to the Lambda execution role (SendSESEmailToContact-role-30r3i2wx). Along with the basic execution role, this grants permissions to access S3 and send emails via SES.

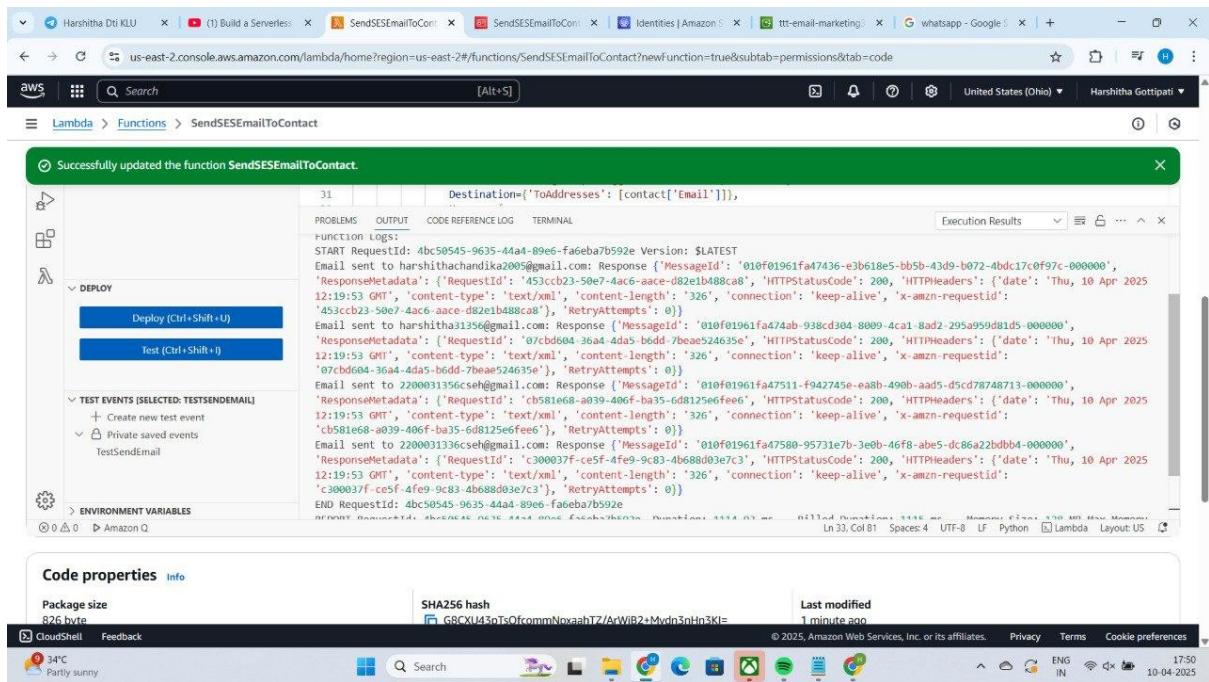


Fig 22: The Lambda function SendSESEmailToContact was successfully updated and tested. The logs confirm that emails were sent to multiple recipients using Amazon SES, with HTTPStatusCode: 200 indicating success

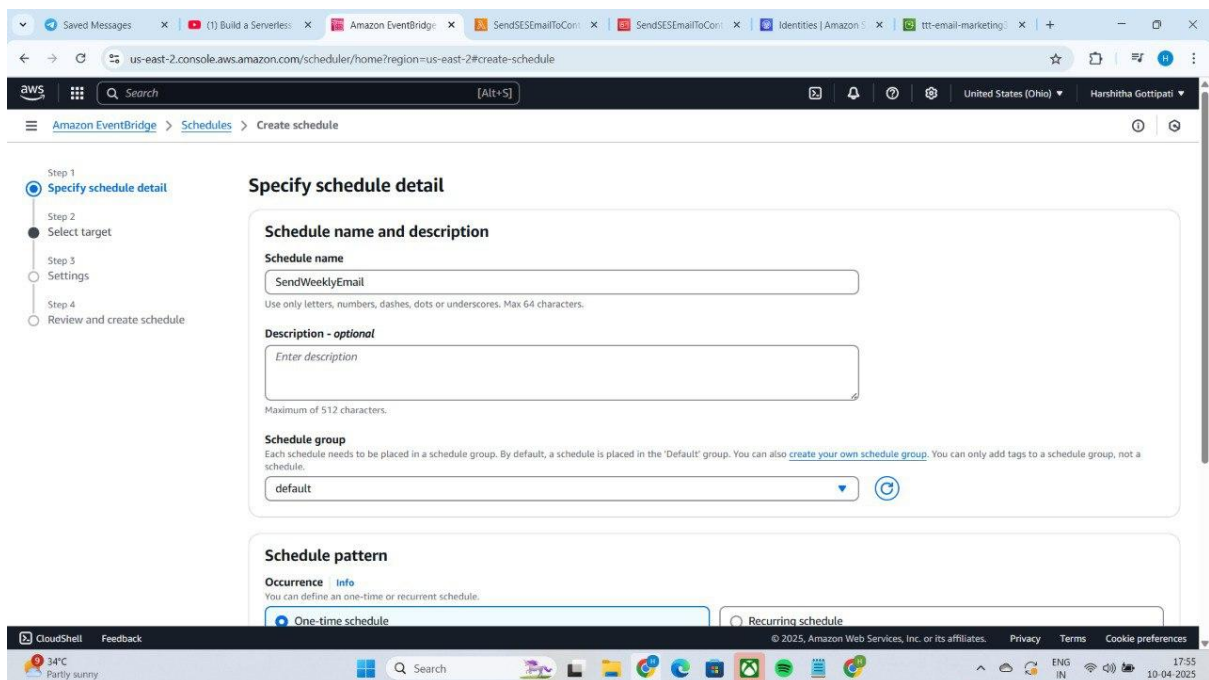


Fig 23: In AWS EventBridge, a new schedule named SendWeeklyEmail is being created. The user is on Step 1: Specify schedule detail, with the schedule being added to the default schedule group. The intention is to trigger a task, possibly a Lambda function, on a recurring basis (e.g., weekly).

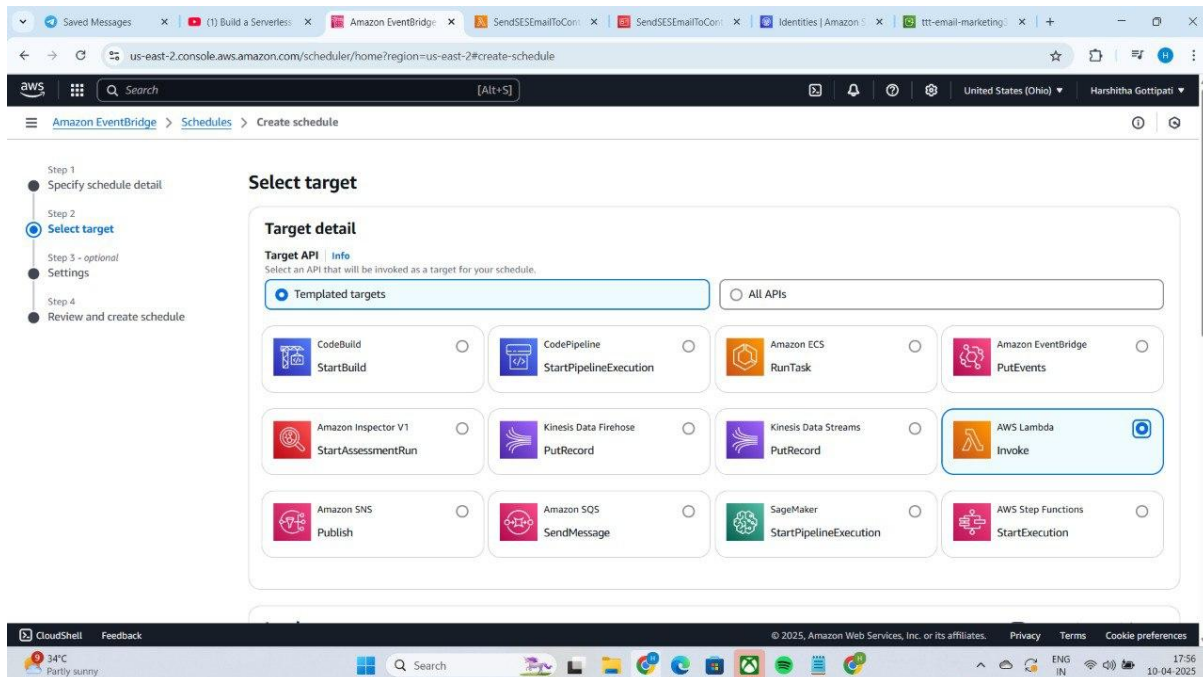


Fig 24: This image shows Step 2: Select target in the AWS EventBridge schedule creation process. The user has chosen AWS Lambda (Invoke) as the target, meaning the scheduled EventBridge rule will trigger a Lambda function execution based on the defined schedule.

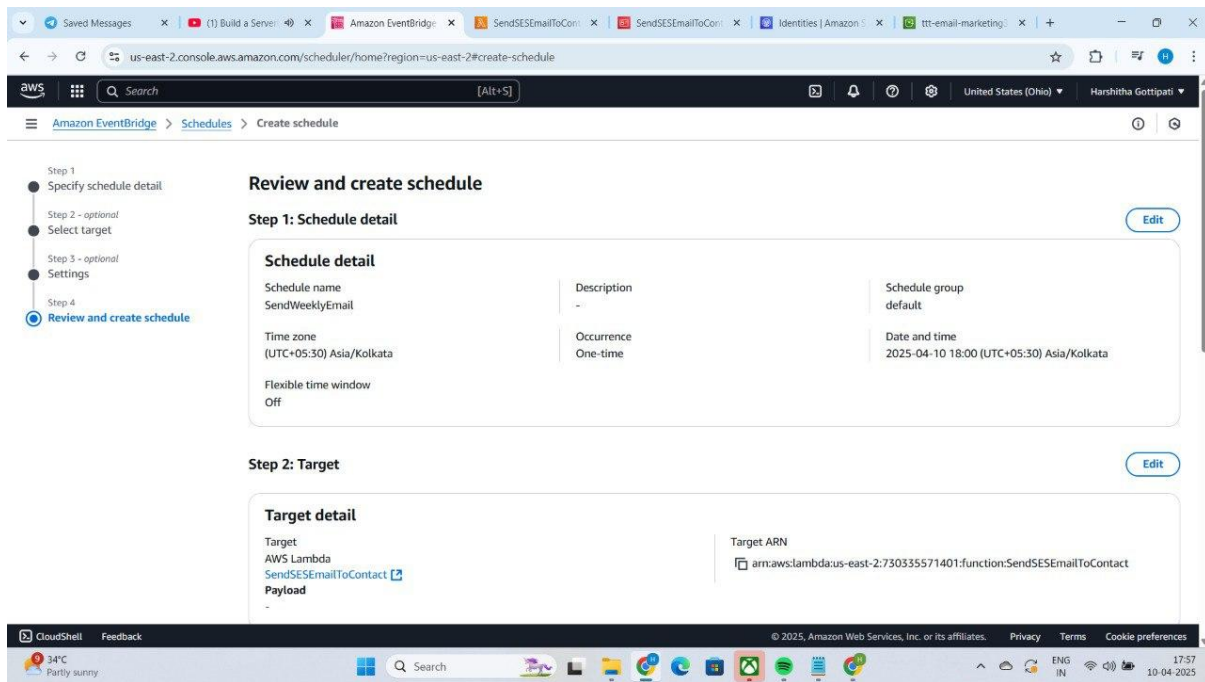


Fig 25: Review and create schedule in Amazon EventBridge. The schedule named **SendWeeklyEmail** is set as a one-time schedule for **April 10, 2025, at 18:00 IST (Asia/Kolkata)**. The target is an AWS Lambda function named **SendSESEmailToContact**, identified by its ARN.

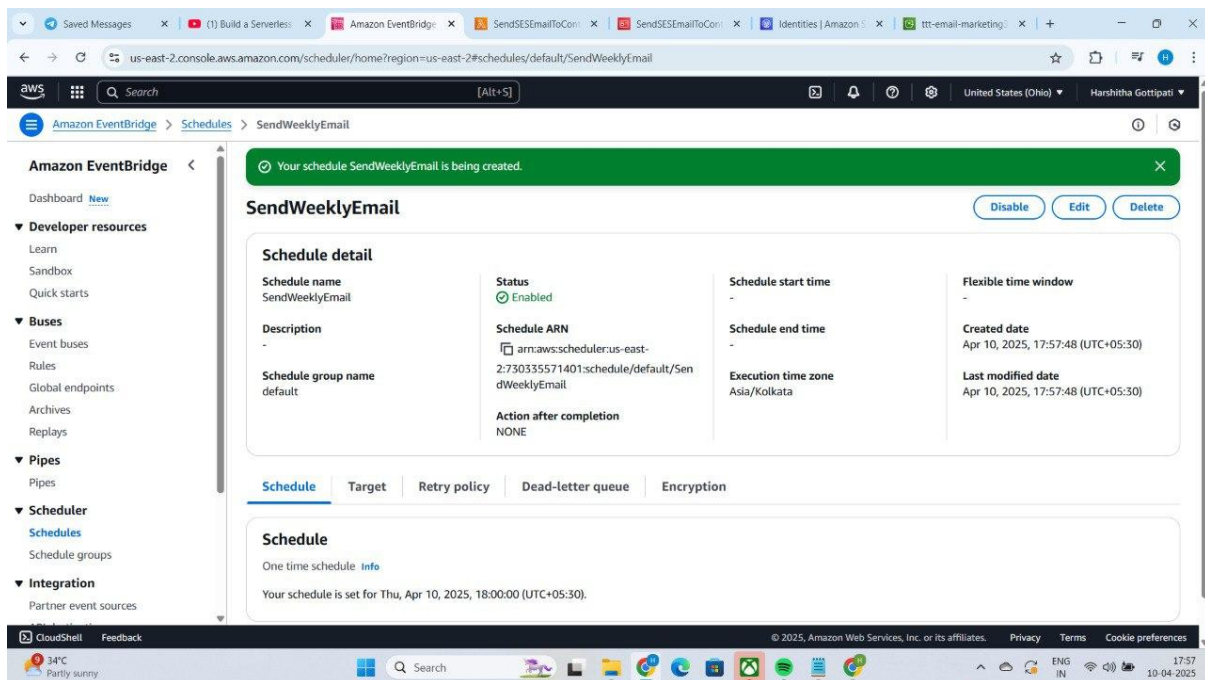


Fig 26: The SendWeeklyEmail schedule successfully created in Amazon EventBridge. The schedule is enabled and belongs to the default schedule group. It is a one-time schedule set for April 10, 2025, at 18:00 IST (UTC+5:30). The Schedule ARN is visible, and options to Disable, Edit, or Delete the schedule are available.

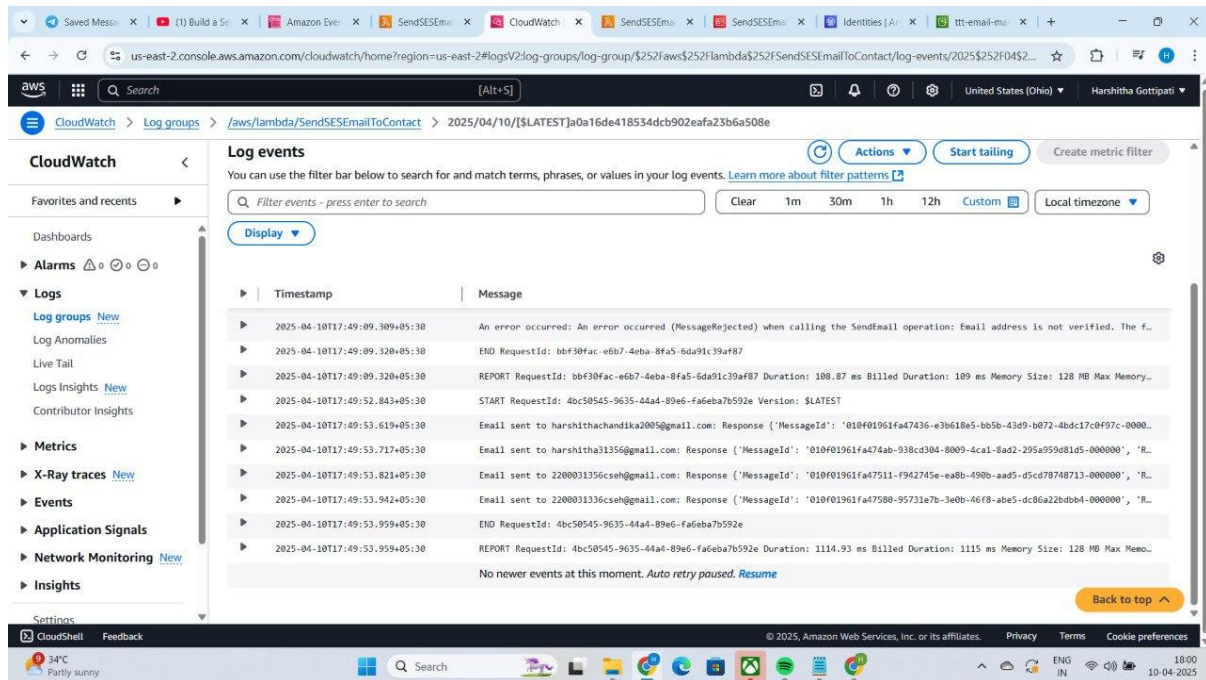
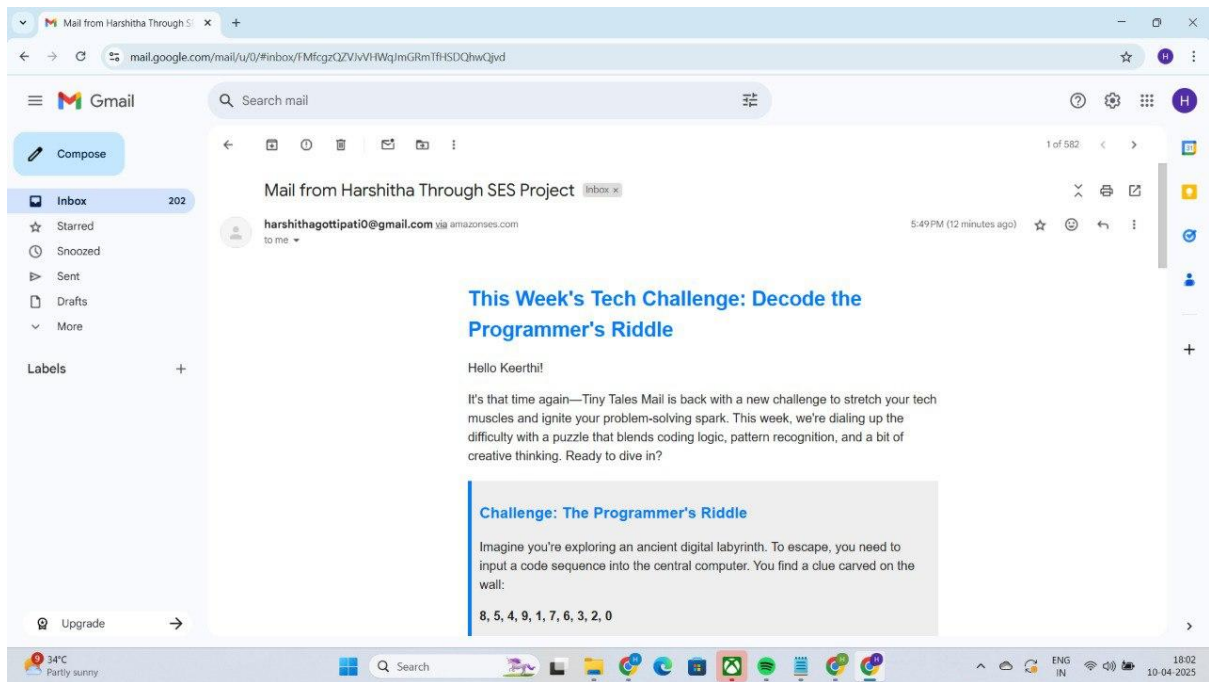


Fig 27: AWS CloudWatch logs for the SendSESEmailToContact Lambda function. It includes:

- Successful email sends to multiple addresses.
- Lambda function execution details like RequestId, duration, memory usage, and timestamps.



This image shows a successfully delivered email from the AWS SES project. The email was sent from harshithagottipati0@gmail.com via Amazon SES to a recipient named Gottipati Harshitha, with the subject: “This Week’s Tech Challenge: Decode the Programmer’s Riddle”

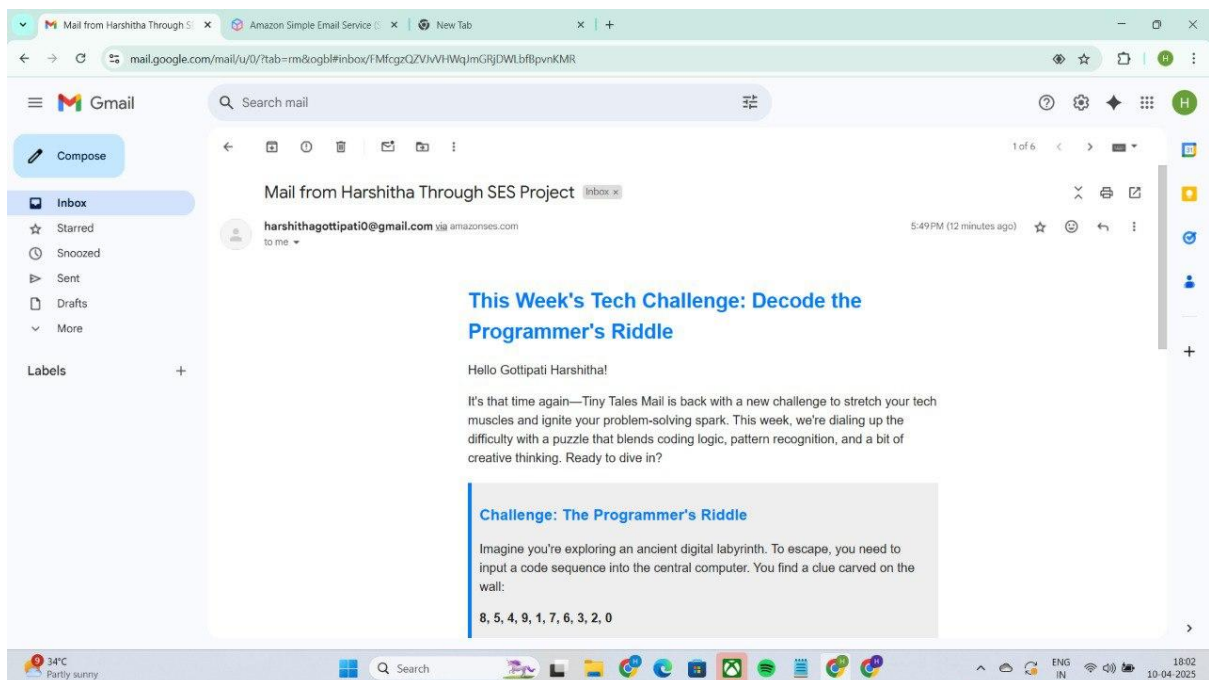


Fig 29: This image shows a successfully delivered email from the AWS SES project. The email was sent from harshithachandika2005@gmail.com via Amazon SES to a recipient named Harshitha, with the subject:

“This Week’s Tech Challenge: Decode the Programmer’s Riddle”

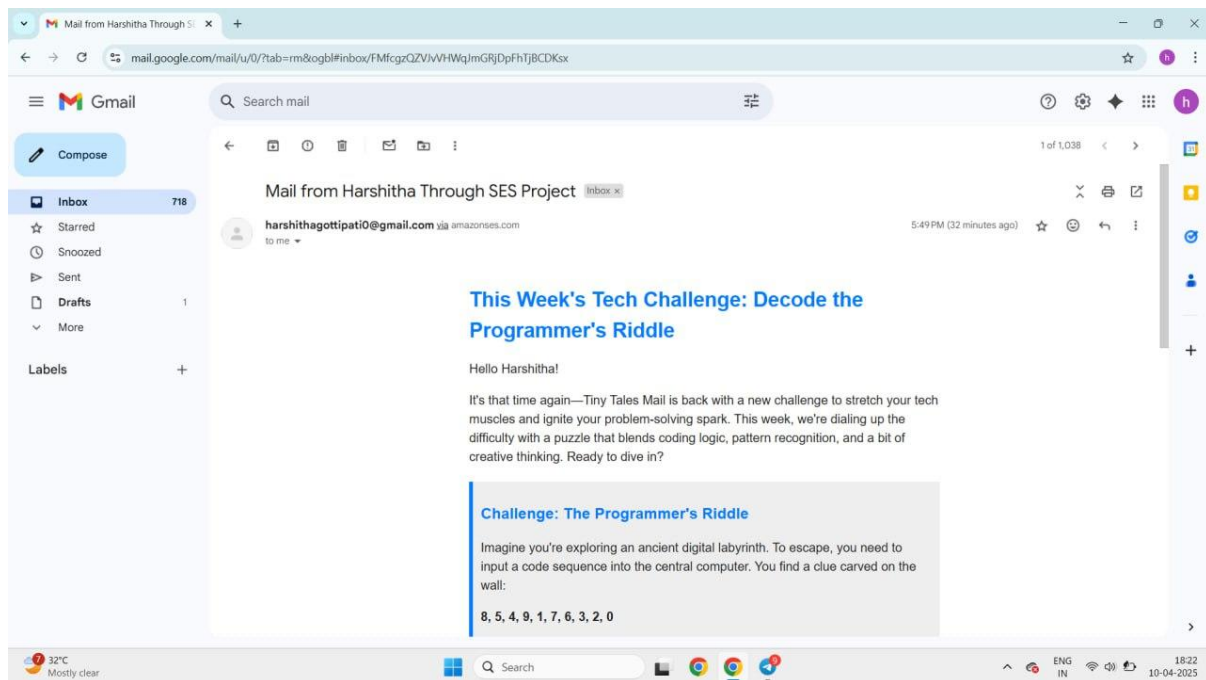


Fig 30: This image shows a successfully delivered email from the AWS SES project. The email was sent from harshitha31356@gmail.com via Amazon SES to a recipient named Keerthi, with the subject:

“This Week’s Tech Challenge: Decode the Programmer’s Riddle”

Discussions & Results

The Serverless Mass Emailing System was successfully implemented using AWS's event-driven architecture. The workflow leveraged Amazon S3 for file storage, Amazon EventBridge for triggering events, AWS Lambda for backend logic execution, and Amazon Simple Email Service (SES) for sending emails in bulk. The system was tested by uploading sample CSV files containing recipient information, simulating real-world marketing and notification use cases.

Upon uploading a CSV file to S3, EventBridge promptly invoked the Lambda function, which parsed each row for recipient email addresses and personalized the email content accordingly. These messages were then dispatched using SES, confirming efficient end-to-end automation of the mass emailing process.

Amazon CloudWatch was integrated for logging and monitoring, enabling real-time visibility into Lambda function executions and SES delivery statuses. Any errors, such as improperly formatted email addresses or delivery failures, were logged for further analysis, ensuring system transparency and reliability.

This serverless setup resulted in:

- **Zero server management overhead**
- **Rapid deployment and scalability**
- **High cost-efficiency**, as resources were consumed only during execution
- **Secure operations** through tightly scoped IAM roles and encryption in transit

The system's modular architecture makes it adaptable for a wide range of use cases—transactional emails, newsletters, event updates, and alerts—across industries.

Conclusion

The Serverless Mass Emailing System built using AWS Lambda and Amazon SES provides a highly scalable, secure, and cost-effective solution for bulk email automation. By combining Amazon S3, EventBridge, Lambda, and SES, the system offers real-time, customizable email delivery without the need for dedicated servers or manual intervention.

The architecture follows best practices for serverless design, including event-driven workflows, stateless functions, and minimal IAM privileges. This ensures not only performance and scalability but also operational simplicity and enhanced security.

This implementation showcases how AWS's serverless suite can be used to simplify traditionally complex tasks such as mass email distribution, making it ideal for startups, enterprises, and developers alike.

Future Work

Enhancements to the serverless mass emailing system can focus on personalization, analytics, and multi-channel communication:

- **Advanced Personalization:** Integrate Amazon Personalize or leverage user metadata to send behavior-based and dynamic content.
- **Analytics Dashboard:** Use Amazon QuickSight or CloudWatch dashboards to visualize email open rates, bounce rates, and engagement metrics.
- **Multi-language Support:** Dynamically generate localized emails based on user region or preferences stored in the metadata.
- **Template Management:** Incorporate Amazon SES Template API or Amazon S3 hosted templates for consistent branding and layout control.
- **Step Functions Integration:** Use AWS Step Functions to handle retries, throttling, or multi-stage workflows (e.g., approval before dispatch).
- **SMS Integration:** Extend the system by integrating with Amazon SNS to send alerts via SMS alongside email, supporting omnichannel communication.

References

1. Amazon Web Services – Getting started with AWS Lambda
<https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>
2. Amazon Simple Email Service (SES) Documentation
<https://docs.aws.amazon.com/ses/latest/dg/Welcome.html>
3. Amazon EventBridge – Event-Driven Workflows
<https://docs.aws.amazon.com/eventbridge/latest/userguide/what-is-amazon-eventbridge.html>
4. AWS Identity and Access Management (IAM) Best Practices
<https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
5. Amazon CloudWatch – Monitoring AWS Resources and Applications
<https://docs.aws.amazon.com/cloudwatch/latest/monitoring/WhatIsCloudWatch.html>
6. Boto3 – AWS SDK for Python
<https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>